

Diffusion Model Notebook Report

By: Erick Banegas

Introduction

In this notebook, I implemented a diffusion-based generative model from scratch to understand how modern generative AI systems create images. The main objective of this project was to learn how diffusion models work, from the process of adding noise to images to training a neural network capable of reversing that process and generating new images.

Through this hands-on exercise, I explored key concepts such as noise scheduling, U-Net architectures, time embeddings, and evaluation using CLIP. This project allowed me to bridge theoretical concepts with practical implementation using PyTorch.

Environment Setup and Reproducibility

To begin, I installed and imported all the necessary libraries, including PyTorch and supporting tools. I ensured reproducibility by setting a fixed random seed for both PyTorch and NumPy. This step is critical because diffusion models are highly sensitive to initialization, and reproducibility helps with debugging and consistent experimentation.

I also configured the computation device, allowing the model to run on GPU if available, which significantly improves training performance.

Dataset Selection and Preparation

For this experiment, I selected a dataset suitable for my computational resources. The notebook provides flexibility to choose between datasets such as MNIST, Fashion-MNIST, or CIFAR-10.

After selecting the dataset, I validated its configuration and inspected its properties, including shape, data type, and class distribution. I used PyTorch DataLoaders to efficiently handle batching and sampling during training.

Model Architecture Design

I implemented a U-Net-based architecture, which is particularly effective for diffusion models due to its ability to preserve spatial information through skip connections.

The architecture includes several key components:

- **GELUConvBlock**: A convolutional block with normalization and activation.
- **DownBlock**: Reduces spatial dimensions while extracting important features.
- **UpBlock**: Reconstructs the image while combining features from earlier layers.
- **SinusoidalPositionEmbedBlock**: Encodes time-step information for the diffusion process.
- **EmbedBlock**: Handles class conditioning.

The U-Net structure allows the model to learn both global and local features, which is essential for accurately removing noise from images.

Diffusion Process Implementation

I implemented both the forward and reverse diffusion processes.

Forward Process:

In this stage, I gradually added Gaussian noise to the images using a predefined noise schedule. This process transforms a clean image into pure noise over multiple steps.

Reverse Process:

The model learns to reverse this process by predicting the noise added at each step. Using this prediction, it reconstructs the original image step by step.

I also visualized how images progressively degrade during the forward process, which helped me better understand how diffusion works.

Training the Model

To train the model, I followed these steps:

1. Sample a clean image.
2. Add noise at a random timestep.
3. Ask the model to predict the noise.

4. Compute the loss using Mean Squared Error (MSE).
5. Update the model parameters using backpropagation.

I also implemented:

- **Gradient clipping** to stabilize training
- **Early stopping** to prevent overfitting
- **Loss tracking** for both training and validation

By monitoring the loss curves, I was able to observe how the model improved over time.

Image Generation

After training, I used the model to generate new images from pure noise. The model progressively refined random noise into recognizable digits.

I also created functions to:

- Generate multiple samples of a specific class
- Visualize the generation process step by step

This helped me understand how the model transforms noise into meaningful structures.

CLIP Evaluation

To evaluate the quality of generated images, I integrated CLIP (Contrastive Language-Image Pre-training).

CLIP allowed me to:

- Measure how well generated images matched their intended labels
- Provide a semantic evaluation of image quality
- Compare different outputs objectively

I also implemented GPU memory management techniques to prevent out-of-memory errors during CLIP evaluation.

Key Insights and Observations

Throughout this project, I gained several important insights:

- Diffusion models are highly effective for generating realistic images.
- The U-Net architecture is essential for preserving spatial details.
- The model learns to denoise rather than directly generate images.
- Training stability is crucial and requires careful tuning.
- Evaluation with CLIP adds a powerful semantic layer to model assessment.

Challenges Encountered

During this project, I faced several challenges:

- Understanding the mathematical formulation of diffusion processes
- Implementing the noise schedule correctly
- Managing GPU memory during training and evaluation
- Debugging model architecture issues (such as redundant blocks)
- Ensuring stable training without divergence

Each of these challenges helped deepen my understanding of deep learning systems.

Applications of Diffusion Models

This type of model has many real-world applications, including:

- Image generation for art and design
- Data augmentation for machine learning
- Medical imaging reconstruction
- Content creation in media and entertainment
- AI-assisted design tools

Diffusion models are currently one of the most powerful approaches in generative AI.

Conclusion

In this notebook, I successfully built and trained a diffusion model capable of generating images from noise. This project gave me a deeper understanding of generative AI and the underlying mechanisms of diffusion models.

By combining theory with implementation, I was able to explore advanced concepts such as U-Net architectures, noise scheduling, and CLIP-based evaluation. Overall, this experience strengthened my skills in deep learning and prepared me for more advanced AI projects.